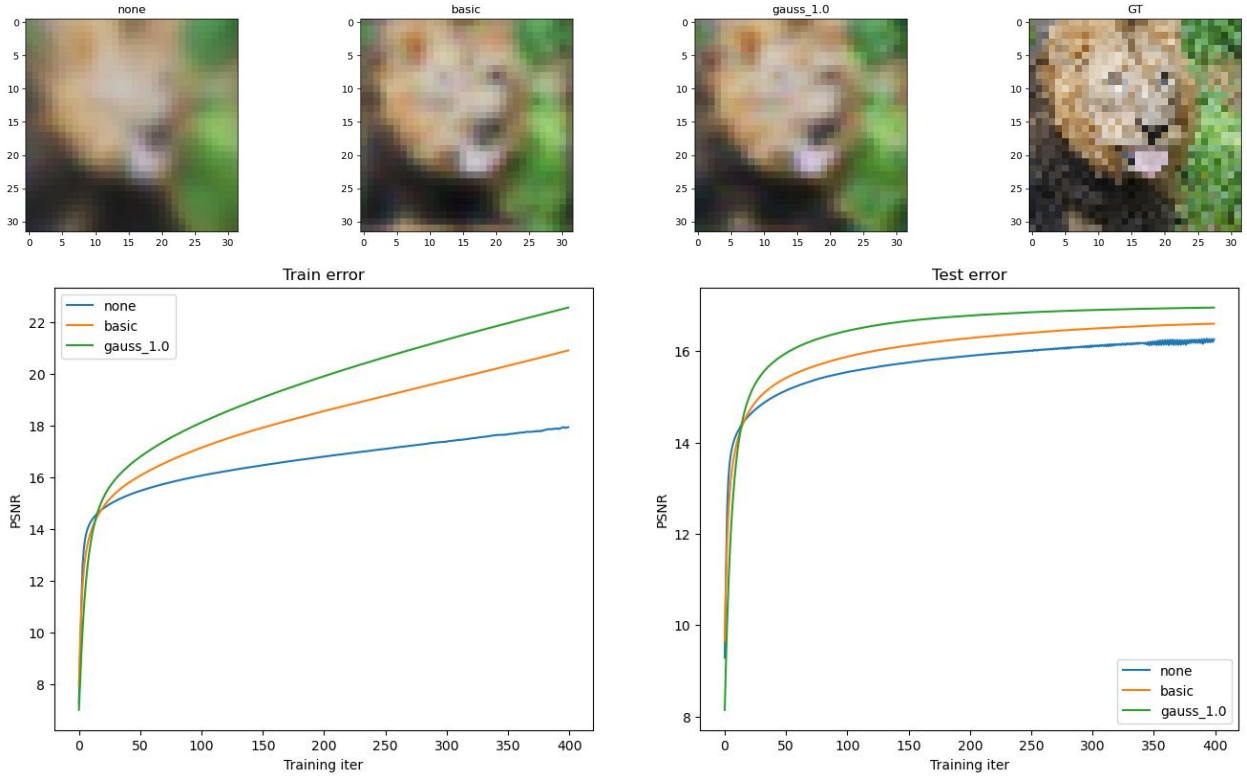
Assignment 2 Report

Name(s):Libin Wang , Licheng Xu
NetID(s):libin2, lx8

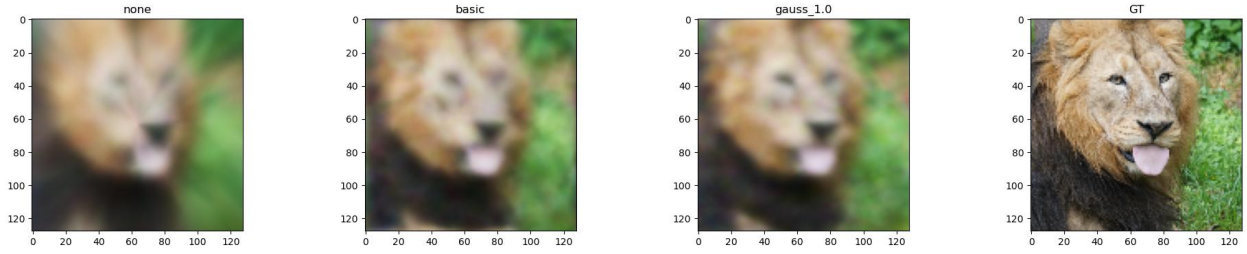
In each the following parts, you should insert the following:

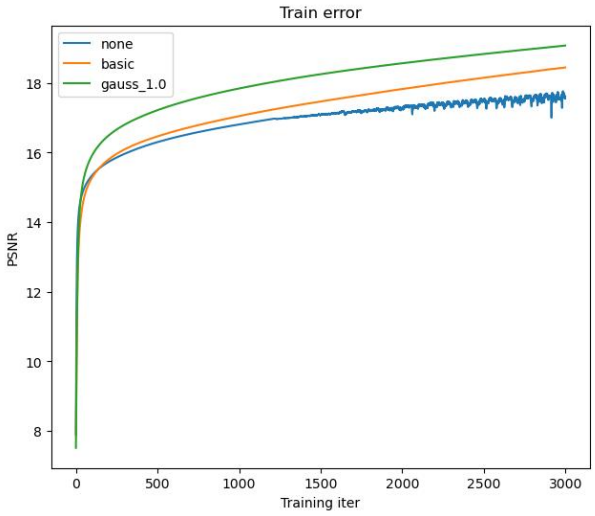
- Train/test loss plots
- Qualitative outputs for GT, No encoding, Basic Positional Encoding, and Fourier Feature Encoding

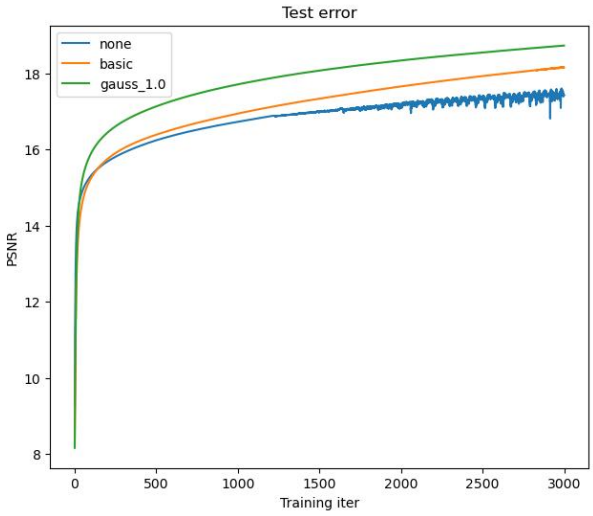
Part 1: Low resolution example



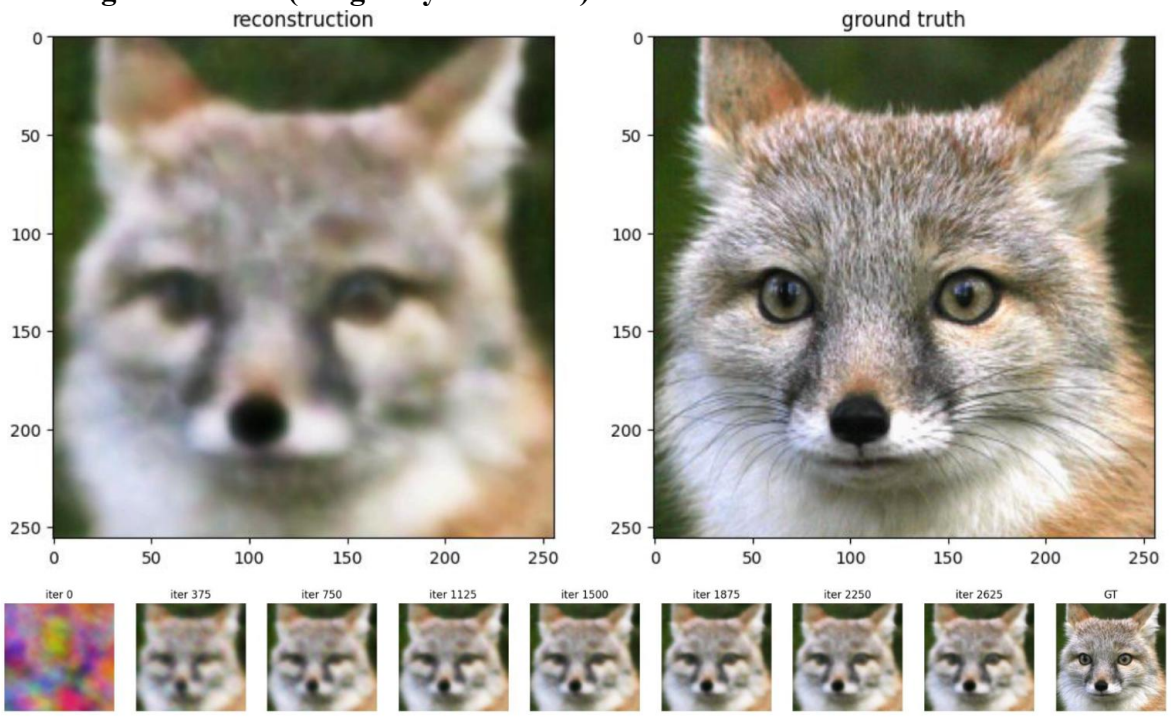
Part 2: High resolution example

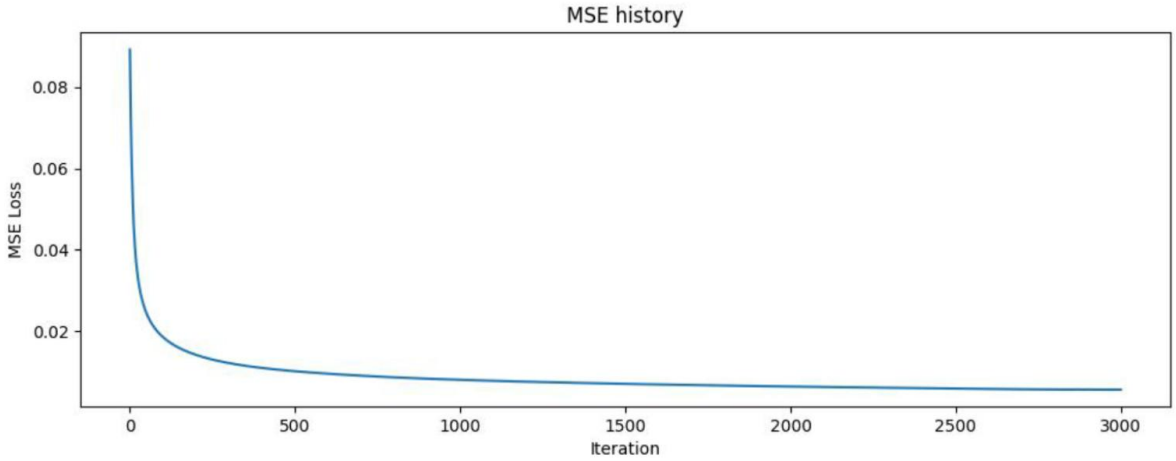


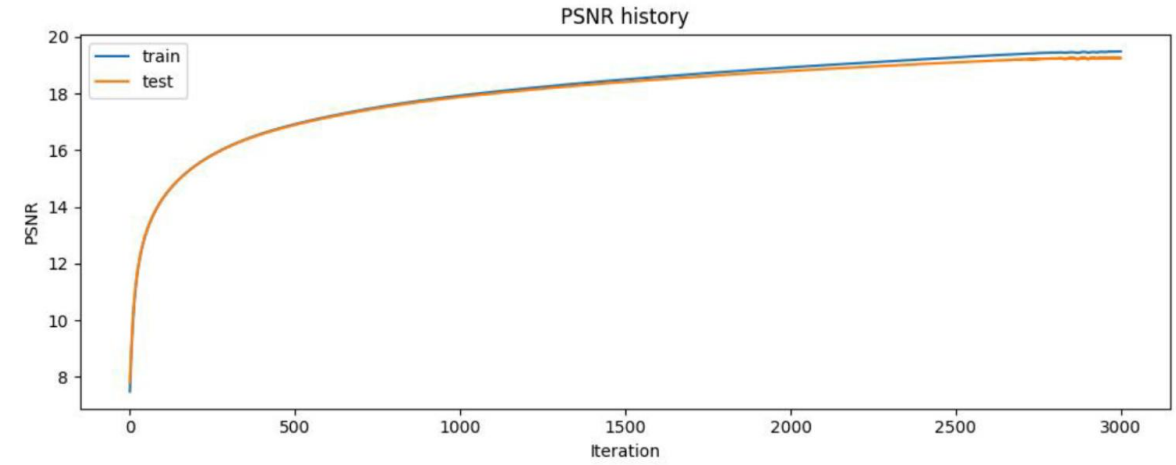




Part 3: High resolution (image of your choice)







(For this part, you can select an image of your choosing and show the performance of your model with the best hyperparameter settings and mapping functions from Part 2. You do not need to show results for all of the mapping functions.)

Part 4: Discussion

Briefly describe the hyperparameter settings you tried and any interesting implementation choices you made.

For the low-resolution task, we initially set hidden_size = 256, epochs = 1000, and lr = 0.7, and observed significant oscillations. We then reduced the learning rate to 0.01, which resulted in very slow convergence. Changing the learning rate to 0.1 caused overfitting and oscillations around 400 epochs. We also experimented with different hidden sizes: using 128 did not yield satisfactory performance, and increasing to 512 showed no improvement over 256. Therefore, we selected 256 as the hidden_size. The final parameters for the low-resolution task are hidden_size = 256, epochs = 400, and lr = 0.1.

For the high-resolution task, we started with the same parameters as the low-resolution task, but the model did not converge. Increasing the epochs to 1000 still did not achieve convergence, so we eventually raised the epochs to 3000. At that point, the model began to oscillate and overfit, but the training time was more reasonable. The final parameters for the low-resolution task are hidden_size = 256, epochs = 3000, and lr = 0.1.
And we use batch normalization to improve the model performance. Before incorporating Batch Normalization, the model did not meet expectations—the accuracy was lower than desired and the training process was unstable. However, after adding Batch Normalization, the model's stability improved significantly, resulting in a convergence rate and final accuracy that met our requirements.

How did the different choices for coordinate mappings functions compare?

Basic and gauss_1.0 reduce the convergence rate during the initial epoch, but eventually achieve higher PSNR for both low-resolution and high-resolution tasks. In the initial epoch, None converges the fastest, followed by Basic and then gauss_1.0. However, the final PSNR is highest for gauss_1.0, followed by Basic, and then None.

Do you make any interesting observations from the train and test plots?

For low-resolution tasks, the training PSNR increases very rapidly, while the test PSNR remains nearly constant. This phenomenon is less pronounced in the high-resolution example, indicating that overfitting is more likely to occur in low-resolution scenarios. Additionally, if the learning rate is too high, both the training and test PSNR plots oscillate. The model without any modifications (None) oscillates more compared to the basic and gauss_1.0 versions. In both training and test plots, the gauss_1.0 and basic versions initially slow down the convergence rate but ultimately achieve a higher PSNR.

What insights did you gain from your own image example (Part 3)?

The image is more detailed than the lion example, and its fur is alternating black and white, which likely increases the difficulty in training and testing. As a result, our model struggles with accurately capturing the fur details. And instead of having white and black pixels alternating, the result has grey pixels.

Part 5: Extra Credit (optional)

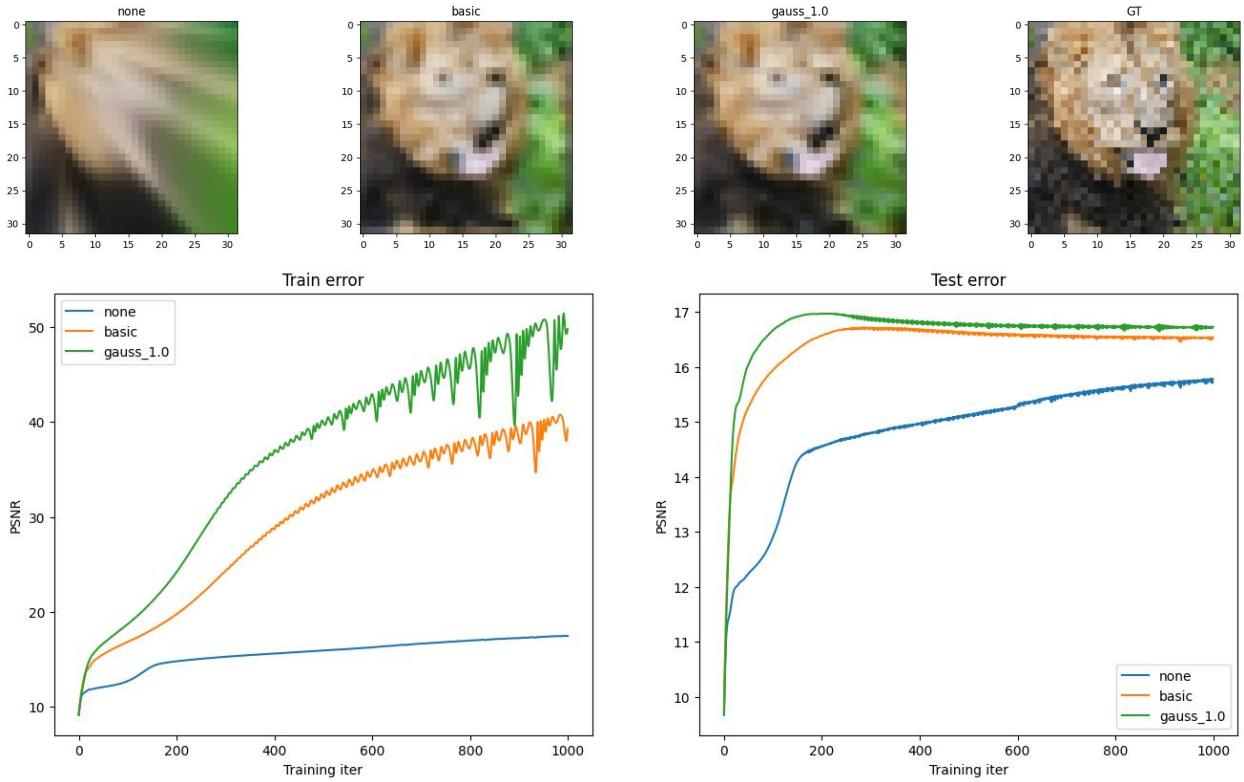
Explain anything you did in detail and provide output images.

***Adam optimizer*

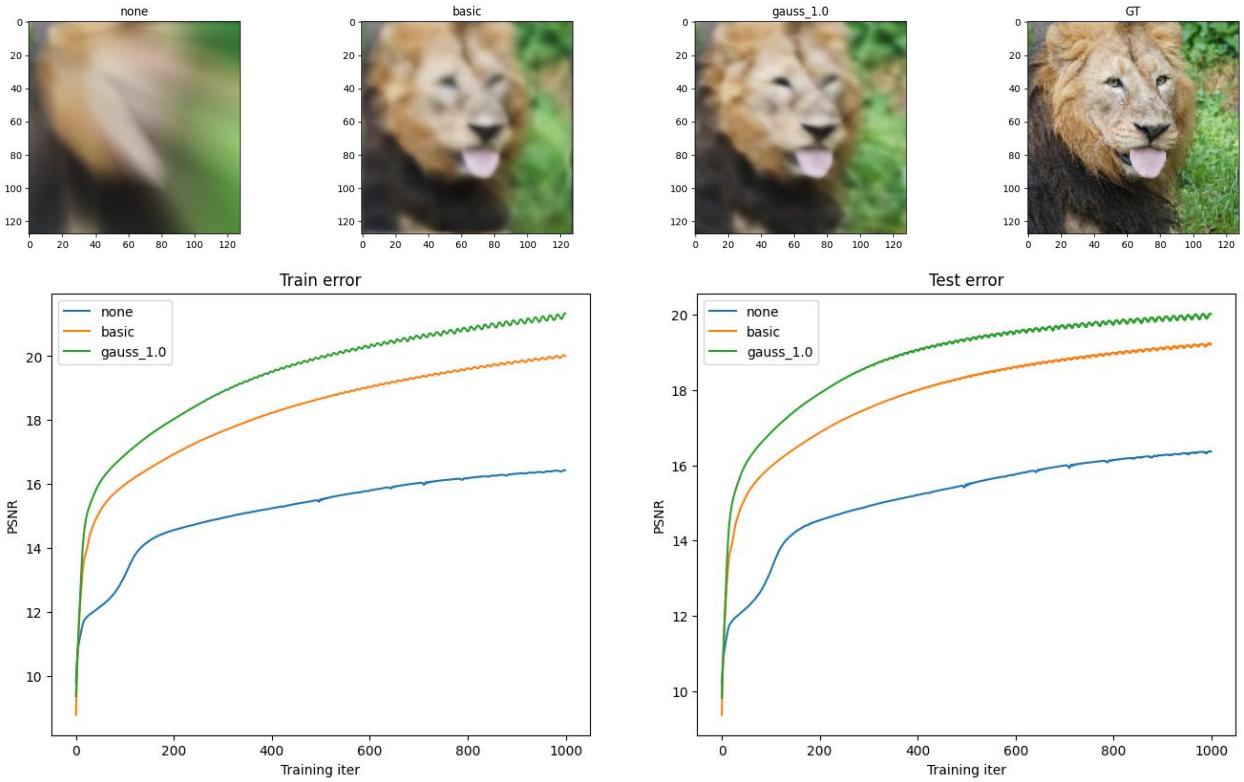
We follow the standard Adam update rule to implement an Adam optimizer in the code. It maintains first-moment estimates (m) and second-moment estimates (v) for each parameter, including weights (W), biases (b), and batch normalization parameters (bn_gamma, bn_beta). At each update step, the first and second moments are computed. These moments are then bias-corrected using the time-step (t) to avoid initialization bias. Finally, the parameters are updated using the Adam formula, where the learning rate is adjusted by dividing the first-moment estimate by the square root of the second-moment estimate plus a small epsilon to ensure numerical stability.

We reconstructed both low-resolution and high-resolution images using the Adam optimizer (without batch normalization). We found that the quality of reconstruction improved, and the PSNR values increased. Additionally, the optimal learning rate for Adam was lower, and the speed of convergence was significantly faster than that of the SGD optimizer.

Low resolution example:

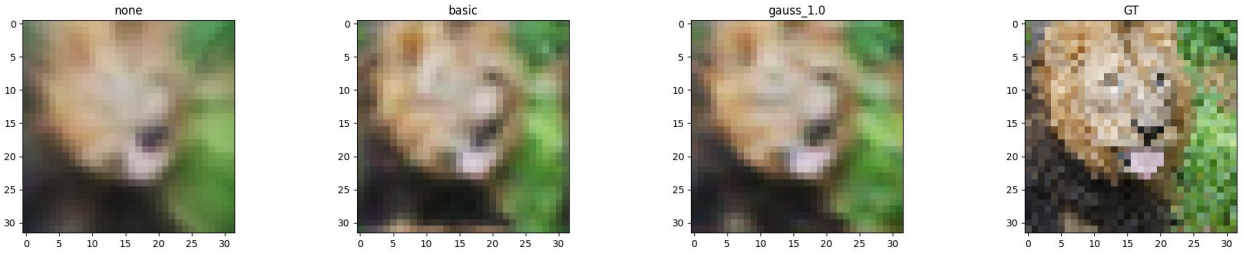


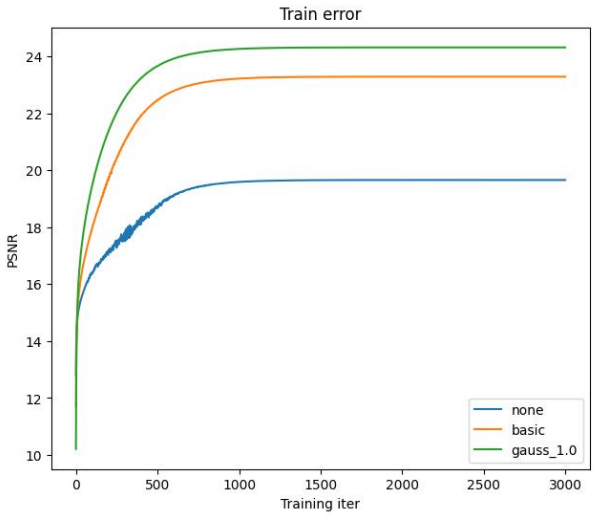
High resolution example:

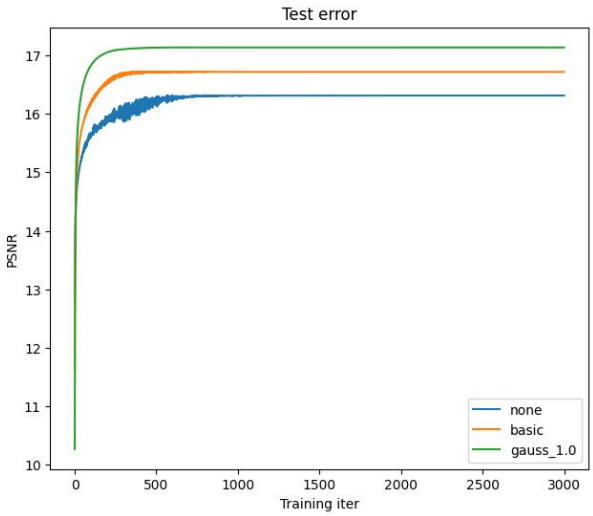


**Pytorch implementation*: We also implemented a PyTorch version of the same network model using SGD optimizer. The code implementation and backpropagation computation are much simpler than in the NumPy version since torch.nn provides many built-in functions, such as nn.Linear(), nn.ReLU(), and nn.Sigmoid(). This allows us to compute the gradients for each weight and bias using autograd. Moreover, the PyTorch model can use CUDA and GPU acceleration, making the computations significantly faster.

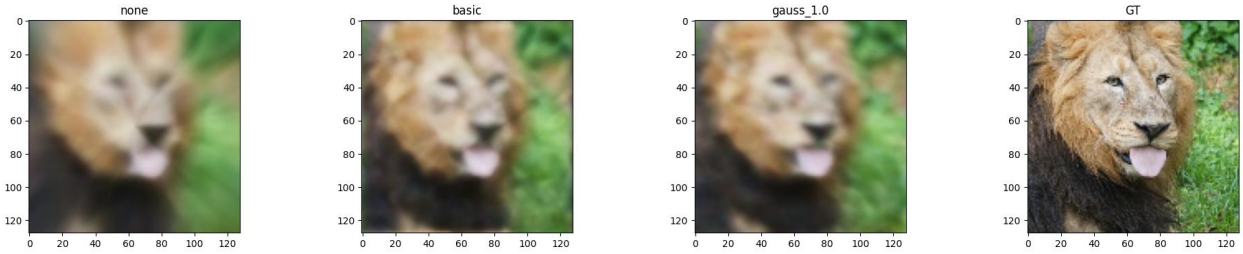
Low resolution example:

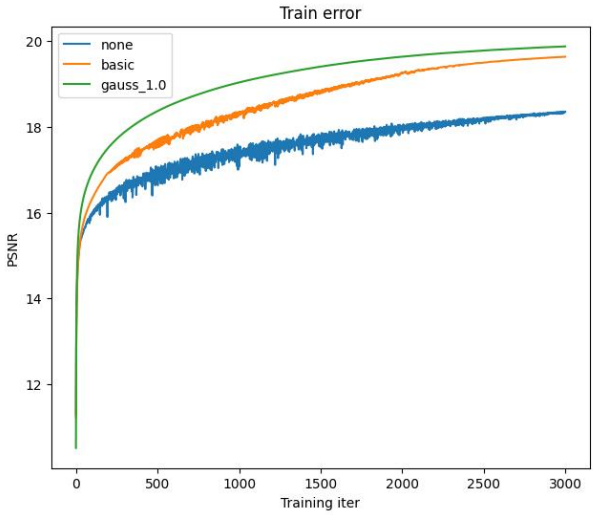


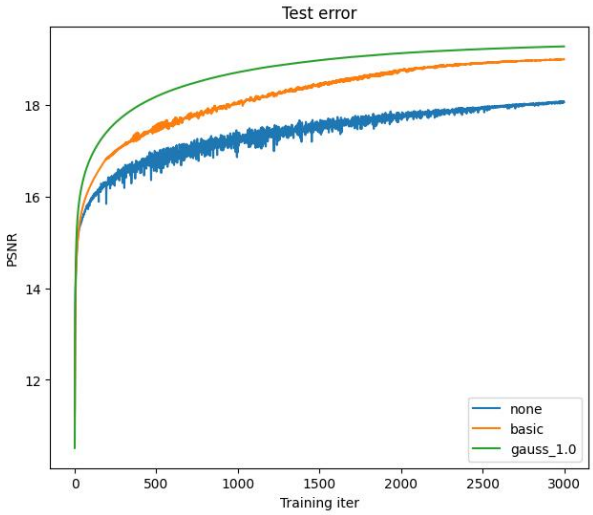




High resolution example:







For both reconstructions, we used the same number of epochs = 3000, the same learning rate = 0.3, and the same hidden layer size = 256. Additionally, we implemented learning rate decay (by a constant factor) in the PyTorch model, and applied a stronger decay for the low-resolution example to mitigate oscillations in the PSNR curve. Compared to the NumPy version, the quality of the reconstructed images was nearly identical, as the PSNR values for the NumPy and PyTorch models were almost the same.

Batch normalization

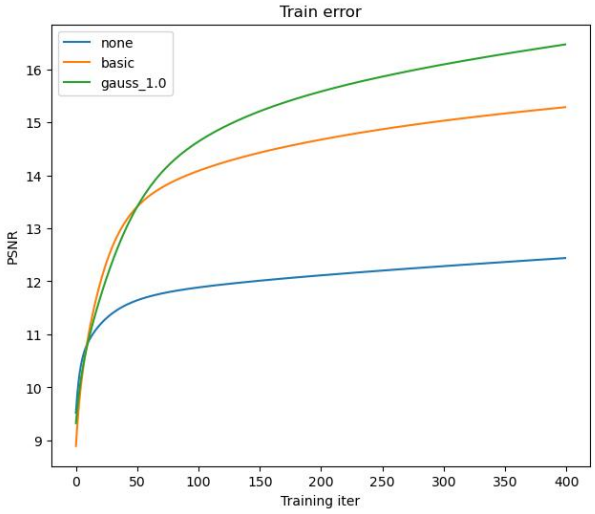
We have added batch normalization to the original neural network.

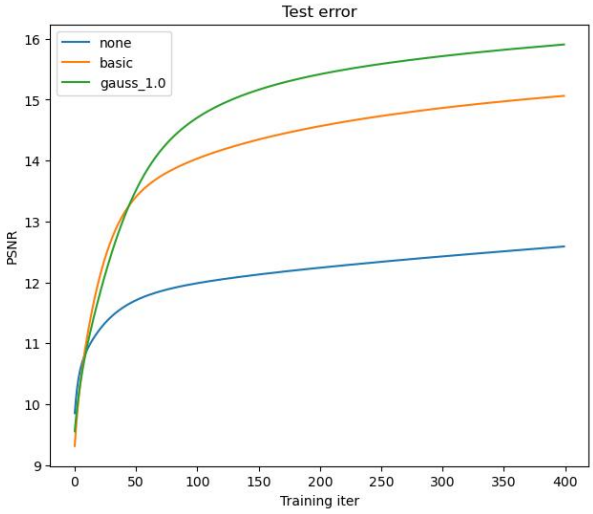
BN Forward Pass:
This function computes the mean, variance, and standard deviation for a layer's output. It then normalizes the output, scales it with γ (gamma), and shifts it with β (beta).

BN Backward Pass:
A corresponding bn_backward function was added to compute gradients with respect to the input of the BN layer. It retrieves the cached intermediate values (mean, variance, standard deviation, normalized input) from the forward pass.

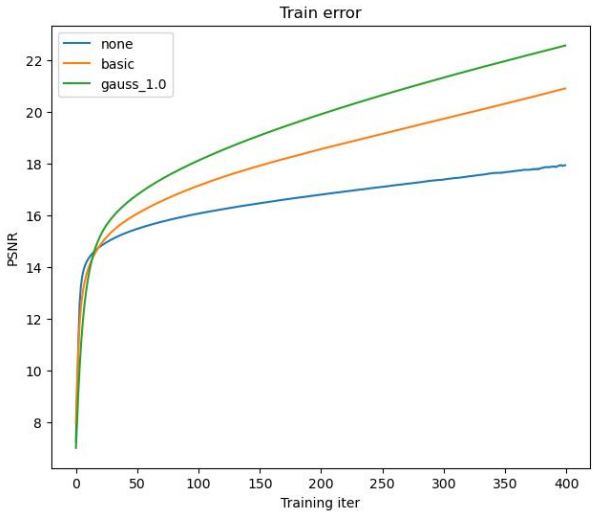
Revision of the Previous Part:
We have added parameter updates for the batch normalization parameters and integrated bn_forward and bn_backward into the existing forward and backward code.

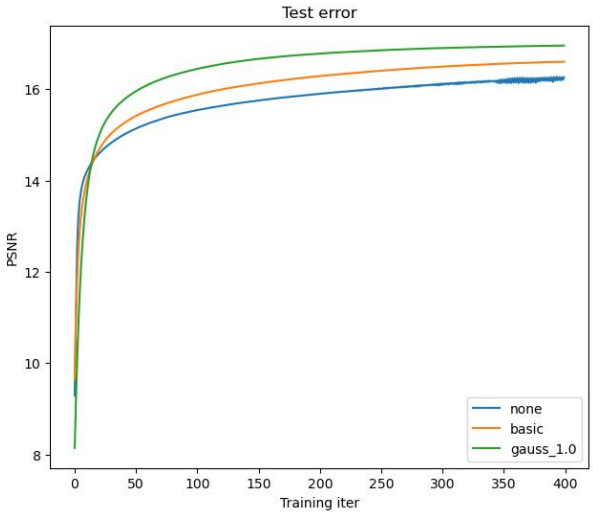
Effect on result
Low resolution part, it greatly increase the PSNR for None mapping.
Without batch normalization:





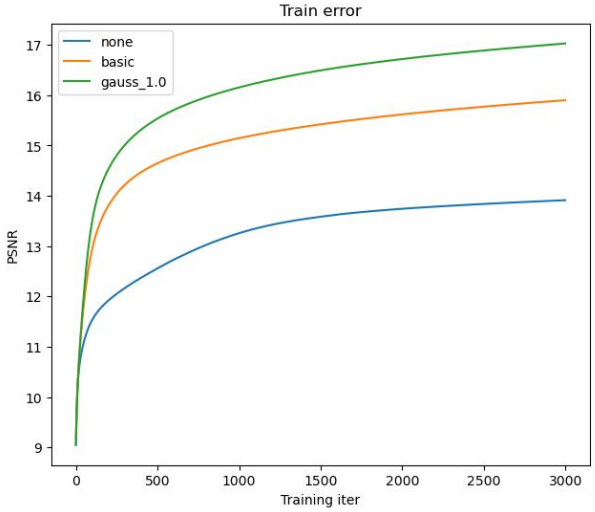
With batch normalization:

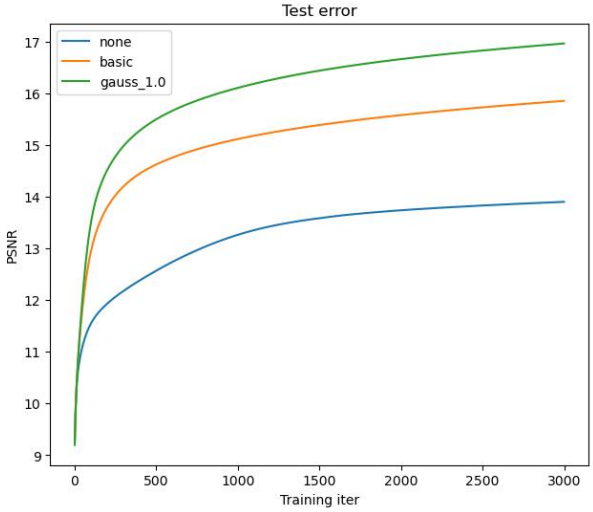




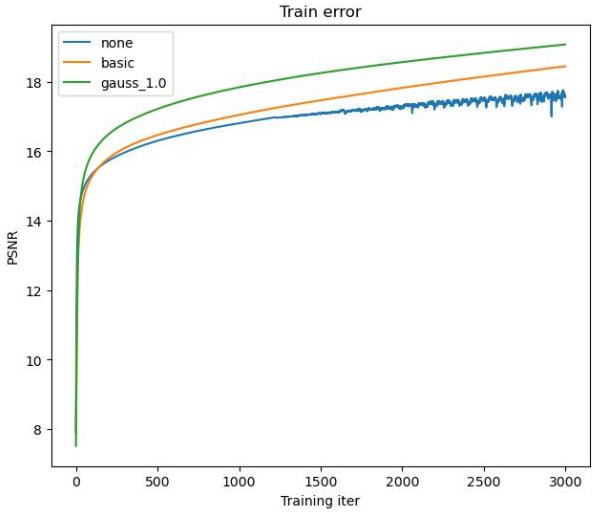
For high resolution part, the PSNR greatly increase and the final image is more clear compare to the result without batch normalization.

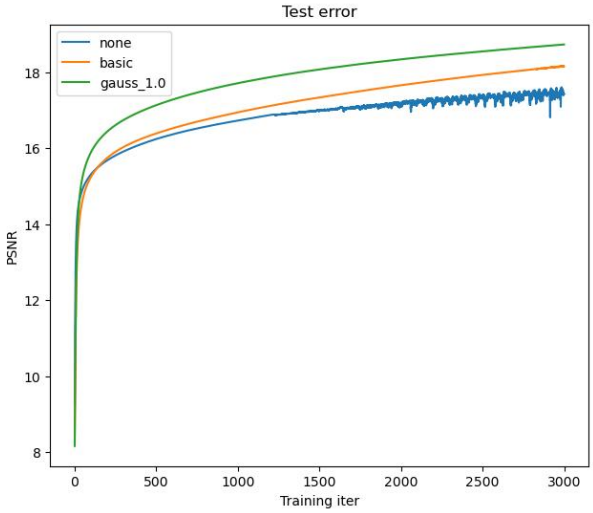
Without batch normalization:



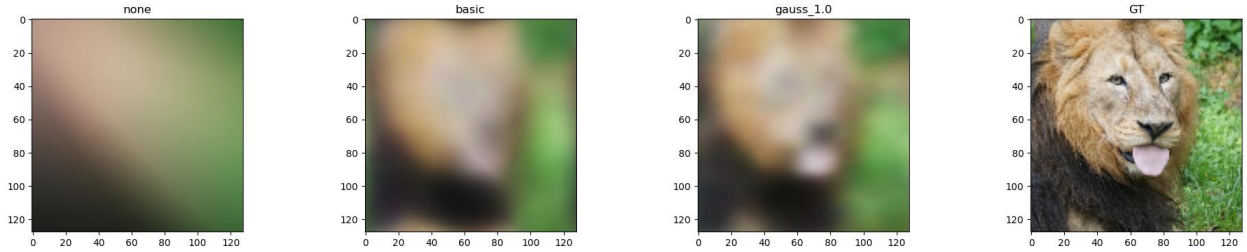


With batch normalization:





Without batch normalization:



With batch normalization: